

作業ログ

報告者 酒井 睦基
報告日 2026 年 6 月 30 日
プロジェクト名 Arduino オーケストラ：きらきら星 輪唱演奏システム

第 1 節 進捗サマリー（要約）

- 全体進捗率：95 [%]
- マイルストーン達成状況：未達成
- 主要成果：
 - サーボモーターと飾り部分の結合が完了
 - ポテンショメータの変化に合わせて各小節毎にテンポが速/遅くなることを確認した
 - PLL 補正によって同期誤差が改善されていることが確認
- 未達成要項：
 - 結合テスト（ソフトウェア部分とハードウェア部分を結合したテスト）
- 重大課題（影響度：低）：7/1（水）の MTG でテストを実施する

第 2 節 作業ログ（詳細トラッキング）

作業項目	開始日	予定完了日	状態	進捗率	依存関係・ブロッカー	コメント
ベビーメ リー風回転 装置制作	6/3	6/23	完了	100%	飾り部分含め ハードウェア が完成した	計画書の想定では第 7～8 週目には終わっている予定であったが、材料調達遅延や不十分な設計、集まれる機会が少なかったことから最終週手前となってしまった

作業項目	開始日	予定完了日	状態	進捗率	依存関係・ブロッカー	コメント
ロードセルの反応検証	6/10	6/23	完了	100%	ロードセルの荷重テスト・キャリブレーションによる閾値の変更まで完了した	小物の重量によつてはシステムが稼働しないためロードセルに乗せる対象の検討を行う
センサ入力・BPM変換処理	6/3	6/23	完了	100%	ポテンシオメータによる可変テンポを確認した	可変テンポに合わせてベビーマリー装置の回転速度が変化することを 7/1 の結合テストで確認する
PLL 補正アルゴリズムの実装	6/10	6/30	完了	100%	PLL 補正によつて同期誤差が大幅に改善されたことを確認した	

第 3 節 技術成熟度レベル TRL の推移

FBS の各要素ごとに現在どのレベルにいるかを以下に示す。

機能	説明	前回 TRL	今回 TRL	備考
I2C 同期 (F1)	マスター→スレーブ同期通信	4	4	マスタ 1 台、スレーブ 4 台の接続を確認できた
音色合成 (F4)	Processing 加算合成	4	4	4 種類の楽器音を判別できるレベルで確認できた
PLL 補正 (F1.3)	位相同期制御	1	4	PLL 補正の有無によつて同期誤差が大きく変化することが確認できた

機能	説明	前回 TRL	今回 TRL	備考
エンタメ機能 (F5)	ロードセル・サーボ 制御	2	4	ロードセルの荷重 検知によってサー ボモーターが稼働 し、ベビーメリー飾 りが回転すること が確認できた
可変テンポ (F2)	BPM 制御	3	4	ポテンショメータに よってテンポが各 小節毎に変化する ことが確認できた
共通ライブ ラリ (F3.1)	PROGMEM + I2C フレーム	4	4	共通ライブラリの 確認と 2 年生間の 共有を確認できた

TRL 定義

- 1 概念レベル：アイデアや原理が確立されており，説明できるレベル
- 2 基本動作レベル：単体で動作確認が可能なレベル
- 3 結合動作レベル：機能を結合し，実際の使用環境に近い環境で動作確認が可能なレベル
- 4 実運用レベル：実際の使用環境で安定的な稼働が確認できるレベル

第 4 節 技術性能指標 TPM の推移

MOP の各要素毎に現在の値の程度を以下に示す．

関連 MOP	指標	目標値	前回値	今回値	備考
MOP1-1	楽器間同期誤差 (クラリネット)	≤ 30 ms	1.35	-0.38	実施済
MOP1-2	楽器間同期誤差 (フルート)	≤ 30 ms	3.07	-0.01	実施済
MOP1-3	楽器間同期誤差 (オルガン)	≤ 30 ms	-1.21	-0.19	実施済
MOP1-4	楽器間同期誤差 (ドラム)	≤ 30 ms	4.11	0.23	実施済

関連 MOP	指標	目標値	前回値	今回値	備考
MOP2-1	I2C SYNC パ ケットロス率 (クラリネット)	0 回	0/50	0/50	実施済
MOP2-2	I2C SYNC パ ケットロス率 (フルート)	0 回	0/50	0/50	実施済
MOP2-3	I2C SYNC パ ケットロス率 (オルガン)	0 回	0/50	0/50	実施済
MOP2-4	I2C SYNC パ ケットロス率 (ドラム)	0 回	0/50	0/50	実施済
MOP3	楽曲識別 (中間 確認)	チーム内 識別	識別可能	識別可能	実施済
MOP4	楽器識別 (中間 確認)	チーム内 識別	識別可能	識別可能	実施済
MOP5	リラクゼーショ ン感	心地よく聞 こえる	—	—	未実施
MOP6	BPM 変更反映 速度	1 小節以内	—	—	未計測
MOP7	PLL 収束速度	変化量 ÷10 小節 以内	—	—	未計測

4.1 現在のガントチャート

現在のガントチャートの状態を図 1 に示す。灰色の箇所は実装が完了したことを示す。

4.2 毎日の作業時間と作業内容

定期的に GitHub を確認し PR がきていないかどうかを確認する。PR がきていた場合はファイルの内容をレビューし、問題がない場合は merge する。基本的な作業時間は GitHub の確認とコードレビューを含め 1 時間程度である。また、ハードウェア設計としてサーボモーターとベビーメリー飾りの接続が完成した。サーボモーターの回転に合わせて飾りも回

Arduinoオーケストラ ガントチャート																
カテゴリ	WBS	作業内容	担当	期間	第4週	第5週	第6週	第7週	第8週	第9週	第10週	第11週	第12週			
ハードウェア	1															
	1.1	機材調達・Arduino×5動作確認	全員	第4～5週												
	1.2	通信回路 (I2C配線)・音声出力回路構築	全員	第5～6週												
	1.3	筐体制作・実装	3年	第6～7週												
	1.4	ペビーマリー風回転装置製作 (サーボ+シャフト+アーム+星飾り)	3年	第7～8週												
	1.5	ロードセルの反応検証	3年	第8週												
ソフトウェア	2															
	2.1	楽器別倍音比率の調査・データ化	全員	第4～5週												
	2.1.1	シリアル通信疎通確認 (115200bps)	山口	第6週												
	2.1.2	フルート音色作成 (倍音+ADSR)	山口	第6～8週												
	2.1.3	クラリネット音色作成 (奇数倍音+LPF)	佐藤	第6～8週												
	2.1.4	オルガン音色作成 (加算合成+ボイス管理)	成毛	第6～9週												
	2.1.5	ドラム音色作成 (ノイズバースト+短エンベロープ)	成毛	第7～8週												
	2.1.6	Serial制御ロジック実装 (5台同時受信・パート振分け)	成毛	第7～9週												
	2.2	センサ入力・BPM変換処理 (EMA+デッドバンド+×10整数化)	成毛	第6～9週												
	2.2.1	BPM変化制限ロジック (1小節最大10BPM)	成毛	第7～9週												
	2.2.2	CONFIG送信処理 (起動時 entry_offset配布)	佐藤・山口	第6～7週												
	2.2.3	SYNCマーカー生成・送信処理 (小節頭配信)	佐藤・山口	第7～8週												
	2.2.4	START/STOP制御処理	佐藤・山口	第7～8週												
	2.2.5	サーボ制御・状態管理実装 (IDLE→PLAYING→FINISHED)	佐藤・山口	第8～10週												
	2.3	共通ライブラリ整備 (PROGMEM+I2Cフレーム)	成毛	第5～6週												
	2.3.1	ISR受信・デコード処理 (onReceive)	佐藤・山口	第6～8週												
	2.3.2	PLL補正アルゴリズム (補正0.3+50msスナップ)	佐藤・山口	第7～10週												
	2.3.3	音符スケジューリング (PROGMEM→Serial)	各楽器担当	第7～9週												
	2.3.4	輪唱位置管理 (my_local_bar計算)	各楽器担当	第7～9週												
統合・評価	3															
	3.1	単体・同期精度テスト (V1聴取+V2 FFT比較)	全員	第8～9週												
	3.2	全体結合演奏テスト (5台接続・12小節完走)	全員	第9～10週												
	3.3	可変テンポ動作検証 (誤差分散±50ms以内)	全員	第10週												
	3.4	全体結合演奏テスト (ハードウェア・ソフトウェアの連結)	全員	第11週												
	3.5	最終発表・報告 (アンケート集計)	全員	第10～12週												

図 1 現在のガントチャート

転し、軸がブレて倒れることがないことも確認した。設計とテストに要した時間はおよそ 3 時間程度である。

第 5 節 課題管理

- 課題：結合テスト (ソフトウェア部分とハードウェア部分の結合)
 - － 発生原因：ハードウェア部分の作成が今週完成したため
 - － 影響度：高
 - － 優先度：高
 - － 対策：MTG で結合テストを実施する
 - － 期限：7/1
 - － 状態：対応中

第 6 節 AI 利用状況と検証 (再現性重視)

今週はハードウェア設計とコードチェックのみを行っていたため AI を利用しなかった。そのため今回の作業ログではこの節の記載を省く。

6.1 利用内容

- 利用目的：
- 入力（プロンプト要旨）：
- 出力概要：

6.2 検証方法

- 検証手法：
—
- 参照資料：
—
- 検証手順：
1. 手順 1：

6.3 ファクトチェック結果

- 一致点：
—
- 不一致・疑義：
—
- 最終判断：
- 根拠：
—

第 7 節 次回計画

- 全員：
 - 結合テスト
 - リラクゼーション感のテスト
 - BPM 変更反映速度のテスト
 - PLL 収束速度のテスト
- 期限：7/1（第 10 週）

第 8 節 その他

- 結合テストが無事完了した場合は評価指標の測定に移る。

第9節 付録

- 添付資料：最終的な開始トリガ部分のコード

```
1  #include "HX711.h" // 作者がBogdan NeculaのHX711系のライブラリをイ
   ンストールする必要がある
2  #include <Servo.h>
3
4  // ロードセルの設定
5  const int DT_PIN = 3;
6  const int SCK_PIN = 2;
7  HX711 scale;
8
9  // サーボモーターの設定
10 Servo servo;
11 const int SERVO_PIN = 9;
12 const float THRESHOLD = 50.0; // この重さ(g)を超えたら回す. 実測
   値に合わせて変更する
13 int MOVE_US = 1480; // 回転速度 (BPMの変更に合わせて可変されるよ
   うにする)
14 const int STOP_US = 1500; // 停止
15
16 void setup() {
17     Serial.begin(9600);
18
19     scale.begin(DT_PIN, SCK_PIN);
20     scale.set_scale(2280); // キャリブレーションで設定した値, 必要が
   あれば変更を加える
21     scale.tare();
22
23     // 初期状態は停止させる
24     servo.attach(SERVO_PIN);
25     servo.writeMicroseconds(STOP_US);
26
27     Serial.println("準備完了");
28 }
29
30 void loop() {
31     float weight = scale.get_units(3); // 3回平均で重さを取得
32
33     // 荷重検知のテスト用のログ
34     Serial.print("重さ: ");
35     Serial.print(weight, 1);
36     Serial.println(" g");
37 }
```

```
38 // ロードセルに小物が置かれているときはサーボモーターを回転させ、  
    小物が取り除かれたら回転を止める  
39 if (weight > THRESHOLD) {  
40     servo.writeMicroseconds(MOVE_US);  
41 } else {  
42     servo.writeMicroseconds(STOP_US);  
43 }  
44 }
```

ソースコード 1 開始トリガシステムのコード

- 添付範囲：すべて